

DNS and Proxy Server Configuration on Fedora Linux

BIND DNS Server and Squid Proxy Server for XYZ.local

Linux Practical tests

NasirAldeen Ishtaiah

Prepared as a practical implementation and documentation of DNS name resolution and proxy access control services.

1. Introduction

This report documents the practical configuration of a DNS server and a proxy server on a Fedora Linux virtual machine for the local company domain XYZ.local. The DNS service was implemented using BIND and was used to resolve local hostnames to IP addresses and perform reverse lookups. The proxy service was implemented using Squid and was used to control access to websites, allow selected networks and hosts, deny social media sites according to the requirements, and demonstrate web caching.

Screenshots are placed directly beside the configuration or test step they prove, instead of being collected at the end. This makes the report easier to follow and connects each result to the exact command or configuration section.

2. Objectives

- Install and configure BIND DNS Server with the XYZ.local zone.
- Configure forward and reverse lookup zones for the required hosts.
- Configure the Fedora system/client DNS resolver.
- Test DNS name resolution using nslookup.
- Install and configure the Squid proxy server.
- Allow subnet 192.168.1.0/24 to access the internet through Squid.
- Allow only two hosts from the 192.168.7.0/24 subnet to access the internet.
- Deny access to facebook.com for all clients.
- Deny access to instagram.com for hosts in subnet 192.168.1.0/24.
- Demonstrate Squid caching using a controlled cacheable HTTP file.

3. Network Design and Addressing

The DNS configuration followed the required local subnet 192.168.64.0/24. For practical proxy browser and curl testing, the Fedora server was reached through its bridged network address 192.168.1.200 because this was the reachable address of the virtual machine in the internet-connected lab environment. The Squid ACL requirements were configured according to the assignment subnets.

3.1 DNS Service Records

Service	Hostname	IP Address
DNS Server	dns.XYZ.local	192.168.64.10
Web Server	web.XYZ.local	192.168.64.20
Mail Server	mail.XYZ.local	192.168.64.30
Proxy Server	proxy.XYZ.local	192.168.64.10
Database Server	db.XYZ.local	192.168.64.50

3.2 Practical Proxy Testing Address

Proxy IP used in browser/curl testing: 192.168.1.200
Proxy port: 3128

4. DNS Server Configuration Using BIND

4.1 Installing BIND and Verifying the Server IP

BIND and the DNS utility tools were installed using dnf. The installed packages were verified using rpm, and the server address was checked using ip -br addr. The Fedora DNS server was assigned 192.168.64.10/24 on interface enp0s3 during the DNS part of the assignment.

```
dnf install bind bind-utils -y
rpm -q bind bind-utils
ip -br addr
```

```
[linuxlab@dns ~]$ mkdir -p ~/XYZ_DNS_Documentation
cd ~/XYZ_DNS_Documentation
[linuxlab@dns XYZ_DNS_Documentation]$ rpm -q bind bind-utils
bind-9.18.26-1.fc38.x86_64
bind-utils-9.18.26-1.fc38.x86_64
[linuxlab@dns XYZ_DNS_Documentation]$ ip -br addr
lo                UNKNOWN    127.0.0.1/8  ::1/128
enp0s3            UP          192.168.64.10/24  fd00::59df:f520:c219:1ccd/64  fe80::3690:2c0d:f9c8:fd0a/64
[linuxlab@dns XYZ_DNS_Documentation]$ grep -nE "listen-on|allow-query|recursion|allow-recursion|zone \"XYZ.local\"|zone \"64.168.192\" /etc/named.conf
grep: /etc/named.conf: Permission denied
```

Figure 1: BIND package verification and Fedora server IP address 192.168.64.10.

4.2 BIND Main Configuration

The file /etc/named.conf was edited so the DNS service would listen on 127.0.0.1 and 192.168.64.10. DNS queries and recursion were allowed for localhost and the 192.168.64.0/24 subnet. The client resolver was also configured to use the local DNS server and the search domain XYZ.local.

```
listen-on port 53 { 127.0.0.1; 192.168.64.10; };
allow-query { localhost; 192.168.64.0/24; };
allow-recursion { localhost; 192.168.64.0/24; };
recursion yes;
```

```
[root@dns XYZ_DNS_Documentation]# cat /etc/resolv.conf
# Generated by NetworkManager
search XYZ.local
nameserver 127.0.0.1
nameserver fd00::3
[root@dns XYZ_DNS_Documentation]# grep -nE "listen-on|allow-query|recursion|allow-recursion|zone \"XYZ.local\"|zone \"64.168.192\" /etc/named.conf
11: listen-on port 53 { 127.0.0.1; 192.168.64.10; };
12: listen-on-v6 port 53 { none; };
13: allow-query { localhost; 192.168.64.0/24; };
20: allow-recursion { localhost; 192.168.64.0/24; };
23: - If you are building an AUTHORITATIVE DNS server, do NOT enable recursion.
25: recursion.
32: recursion yes;
61: zone "XYZ.local" IN {
67: zone "64.168.192.in-addr.arpa" IN {
[root@dns XYZ_DNS_Documentation]#
```

Figure 2: Client resolver configuration and important BIND options from /etc/named.conf.

The forward zone and reverse zone were declared as master zones in /etc/named.conf.

```
zone "XYZ.local" IN {
    type master;
    file "XYZ.local.forward";
    allow-update { none; };
};

zone "64.168.192.in-addr.arpa" IN {
    type master;
    file "XYZ.local.reverse";
    allow-update { none; };
};
```

```
[root@dns XYZ_DNS_Documentation]# tail -n 25 /etc/named.conf
    channel default_debug {
        file "data/named.run";
        severity dynamic;
    };
};

zone "." IN {
    type hint;
    file "named.ca";
};

include "/etc/named.rfc1912.zones";
include "/etc/named.root.key";

zone "XYZ.local" IN {
    type master;
    file "XYZ.local.forward";
    allow-update { none; };
};

zone "64.168.192.in-addr.arpa" IN {
    type master;
    file "XYZ.local.reverse";
    allow-update { none; };
};
[root@dns XYZ_DNS_Documentation]#
```

Figure 3: Full forward and reverse zone declarations in /etc/named.conf.

4.3 Forward and Reverse Zone Files

The forward lookup zone maps hostnames to IP addresses using A records. The reverse lookup zone maps IP addresses back to hostnames using PTR records. These two zone files implement the required DNS table for XYZ.local.

Forward zone: /var/named/XYZ.local.forward

Reverse zone: /var/named/XYZ.local.reverse

```
[root@dns XYZ_DNS_Documentation]# cat /var/named/XYZ.local.forward
$TTL 86400
@      IN      SOA      dns.XYZ.local. admin.XYZ.local. (
                                2026061201 ; Serial
                                1D          ; Refresh
                                1H          ; Retry
                                1W          ; Expire
                                3H )        ; Minimum TTL

; Nameservers
@      IN      NS       dns.XYZ.local.

; Mail servers
@      IN      MX       10 mail.XYZ.local.

; Forward zone
dns    IN      A        192.168.64.10
proxy  IN      A        192.168.64.10
web    IN      A        192.168.64.20
mail   IN      A        192.168.64.30
db     IN      A        192.168.64.50
[root@dns XYZ_DNS_Documentation]# cat /var/named/XYZ.local.reverse
$TTL 86400
@      IN      SOA      dns.XYZ.local. admin.XYZ.local. (
                                2026061201 ; Serial
                                1D          ; Refresh
                                1H          ; Retry
                                1W          ; Expire
                                3H )        ; Minimum TTL

; Nameservers
@      IN      NS       dns.XYZ.local.

; Reverse zone
10     IN      PTR      dns.XYZ.local.
10     IN      PTR      proxy.XYZ.local.
20     IN      PTR      web.XYZ.local.
30     IN      PTR      mail.XYZ.local.
50     IN      PTR      db.XYZ.local.
[root@dns XYZ_DNS_Documentation]# systemctl restart named
```

Figure 4: Forward and reverse zone files for the XYZ.local DNS database.

4.4 DNS Service Validation

The BIND configuration and the two zone files were validated using named-checkconf and named-checkzone. The named service was restarted and verified as active. The firewall was also checked to confirm the DNS service was allowed.

```
named-checkconf
named-checkzone XYZ.local /var/named/XYZ.local.forward
named-checkzone 64.168.192.in-addr.arpa /var/named/XYZ.local.reverse
```

```
systemctl status named
firewall-cmd --list-services
```

```
[root@dns XYZ_DNS_Documentation]# systemctl status named
● named.service - Berkeley Internet Name Domain (DNS)
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─10-timeout-abort.conf
   Active: active (running) since Fri 2026-06-12 15:40:18 EEST; 4s ago
     Process: 2917 ExecStartPre=/bin/bash -c if [ ! "${DISABLE_ZONE_CHECKING}" == "yes" ]; then /usr/bin/named-checkconf -z "$NAMEDCONF"; else echo "Checking"
     Process: 2919 ExecStart=/usr/sbin/named -u named -c ${NAMEDCONF} $OPTIONS (code=exited, status=0/SUCCESS)
    Main PID: 2920 (named)
       Tasks: 12 (limit: 8224)
      Memory: 9.1M
         CPU: 99ms
    CGroup: /system.slice/named.service
            └─2920 /usr/sbin/named -u named -c /etc/named.conf

Jun 12 15:40:18 dns.XYZ.local named[2920]: zone 0.in-addr.arpa/IN: loaded serial 0
Jun 12 15:40:18 dns.XYZ.local named[2920]: zone 64.168.192.in-addr.arpa/IN: loaded serial 2026061201
Jun 12 15:40:18 dns.XYZ.local named[2920]: zone XYZ.local/IN: loaded serial 2026061201
Jun 12 15:40:18 dns.XYZ.local named[2920]: zone localhost.localdomain/IN: loaded serial 0
Jun 12 15:40:18 dns.XYZ.local named[2920]: zone 1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.ip6.arpa/IN: loaded serial 0
Jun 12 15:40:18 dns.XYZ.local named[2920]: zone localhost/IN: loaded serial 0
Jun 12 15:40:18 dns.XYZ.local named[2920]: zone 1.0.0.127.in-addr.arpa/IN: loaded serial 0
Jun 12 15:40:18 dns.XYZ.local named[2920]: all zones loaded
Jun 12 15:40:18 dns.XYZ.local named[2920]: running
Jun 12 15:40:18 dns.XYZ.local system[1]: Started named.service - Berkeley Internet Name Domain (DNS).
[root@dns XYZ_DNS_Documentation]# named-checkconf
[root@dns XYZ_DNS_Documentation]# named-checkzone XYZ.local /var/named/XYZ.local.forward
zone XYZ.local/IN: loaded serial 2026061201
OK
[root@dns XYZ_DNS_Documentation]# named-checkzone 64.168.192.in-addr.arpa /var/named/XYZ.local.reverse
zone 64.168.192.in-addr.arpa/IN: loaded serial 2026061201
OK
[root@dns XYZ_DNS_Documentation]# firewall-cmd --list-services
dhcpv6-client dns mdns samba-client ssh
[root@dns XYZ_DNS_Documentation]# resolvectl flush-caches
```

Figure 5: BIND service status, zone validation results, and firewall DNS service proof.

4.5 Client DNS Configuration and nslookup Testing

The client DNS resolver was configured to use the local DNS server and search domain XYZ.local. The resolver status confirmed that the DNS server was 127.0.0.1 and the domain was XYZ.local.

```
[root@dns XYZ_DNS_Documentation]# resolvectl status
Global
    Protocols: LLMNR=resolve -mDNS -DNSoverTLS DNSSEC=no/unsupported
    resolv.conf mode: foreign
Current DNS Server: fd00::3
    DNS Servers: 127.0.0.1 fd00::3 127.0.0.1
    DNS Domain: XYZ.local

Link 2 (enp0s3)
    Current Scopes: DNS LLMNR/IPv4 LLMNR/IPv6
    Protocols: +DefaultRoute LLMNR=resolve -mDNS -DNSoverTLS DNSSEC=no/unsupported
Current DNS Server: fd00::3
    DNS Servers: 127.0.0.1 fd00::3
    DNS Domain: XYZ.local
```

Figure 6: Resolver status showing the DNS server and DNS domain.

Forward lookup tests resolved the required hostnames to their correct IP addresses.

```
nslookup dns.XYZ.local 127.0.0.1
nslookup web.XYZ.local 127.0.0.1
nslookup mail.XYZ.local 127.0.0.1
nslookup db.XYZ.local 127.0.0.1
nslookup proxy.XYZ.local 127.0.0.1
```

```
[root@dns XYZ_DNS_Documentation]# nslookup dns.XYZ.local 127.0.0.1
nslookup web.XYZ.local 127.0.0.1
nslookup mail.XYZ.local 127.0.0.1
nslookup db.XYZ.local 127.0.0.1
nslookup proxy.XYZ.local 127.0.0.1
Server:      127.0.0.1
Address:     127.0.0.1#53

Name:   dns.XYZ.local
Address: 192.168.64.10

Server:      127.0.0.1
Address:     127.0.0.1#53

Name:   web.XYZ.local
Address: 192.168.64.20

Server:      127.0.0.1
Address:     127.0.0.1#53

Name:   mail.XYZ.local
Address: 192.168.64.30

Server:      127.0.0.1
Address:     127.0.0.1#53

Name:   db.XYZ.local
Address: 192.168.64.50

Server:      127.0.0.1
Address:     127.0.0.1#53

Name:   proxy.XYZ.local
Address: 192.168.64.10
```

Figure 7: Forward lookup tests using nslookup with the local DNS server.

Reverse lookup tests resolved the required IP addresses back to their hostnames. Normal client DNS tests also worked without explicitly specifying 127.0.0.1, proving the client resolver was configured properly.

```
nslookup 192.168.64.10 127.0.0.1
nslookup 192.168.64.20 127.0.0.1
nslookup 192.168.64.30 127.0.0.1
nslookup 192.168.64.50 127.0.0.1
```

```

[root@dns XYZ_DNS_Documentation]# nslookup 192.168.64.10 127.0.0.1
nslookup 192.168.64.20 127.0.0.1
nslookup 192.168.64.30 127.0.0.1
nslookup 192.168.64.50 127.0.0.1
10.64.168.192.in-addr.arpa      name = proxy.XYZ.local.
10.64.168.192.in-addr.arpa      name = dns.XYZ.local.

20.64.168.192.in-addr.arpa      name = web.XYZ.local.

30.64.168.192.in-addr.arpa      name = mail.XYZ.local.

50.64.168.192.in-addr.arpa      name = db.XYZ.local.

[root@dns XYZ_DNS_Documentation]# nslookup web.XYZ.local
nslookup mail.XYZ.local
nslookup db.XYZ.local
nslookup proxy.XYZ.local
Server:      127.0.0.1
Address:     127.0.0.1#53

Name:  web.XYZ.local
Address: 192.168.64.20

Server:      127.0.0.1
Address:     127.0.0.1#53

Name:  mail.XYZ.local
Address: 192.168.64.30

Server:      127.0.0.1
Address:     127.0.0.1#53

Name:  db.XYZ.local
Address: 192.168.64.50

Server:      127.0.0.1
Address:     127.0.0.1#53

```

Figure 8: Reverse lookup tests resolving IP addresses back to hostnames.

```

[root@dns XYZ_DNS_Documentation]# nslookup web.XYZ.local
nslookup mail.XYZ.local
nslookup db.XYZ.local
nslookup proxy.XYZ.local
Server:      127.0.0.1
Address:     127.0.0.1#53

Name:  web.XYZ.local
Address: 192.168.64.20

Server:      127.0.0.1
Address:     127.0.0.1#53

Name:  mail.XYZ.local
Address: 192.168.64.30

Server:      127.0.0.1
Address:     127.0.0.1#53

Name:  db.XYZ.local
Address: 192.168.64.50

Server:      127.0.0.1
Address:     127.0.0.1#53

Name:  proxy.XYZ.local
Address: 192.168.64.10

[root@dns XYZ_DNS_Documentation]#

```

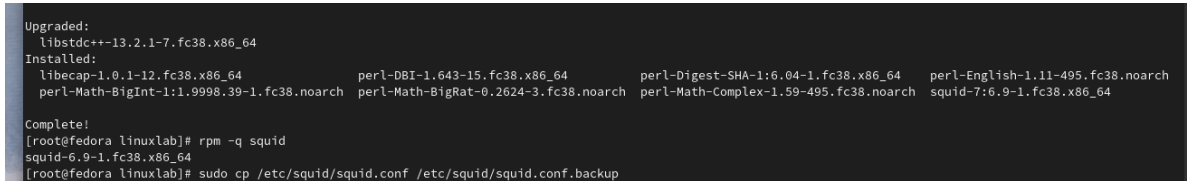
Figure 9: Normal client DNS tests without explicitly specifying the DNS server.

5. Proxy Server Configuration Using Squid

5.1 Installing Squid

Squid was installed using dnf. Some external repository metadata warnings appeared from unrelated repositories, but the Squid package installed successfully. The installed package was verified using rpm.

```
dnf install squid -y
rpm -q squid
```



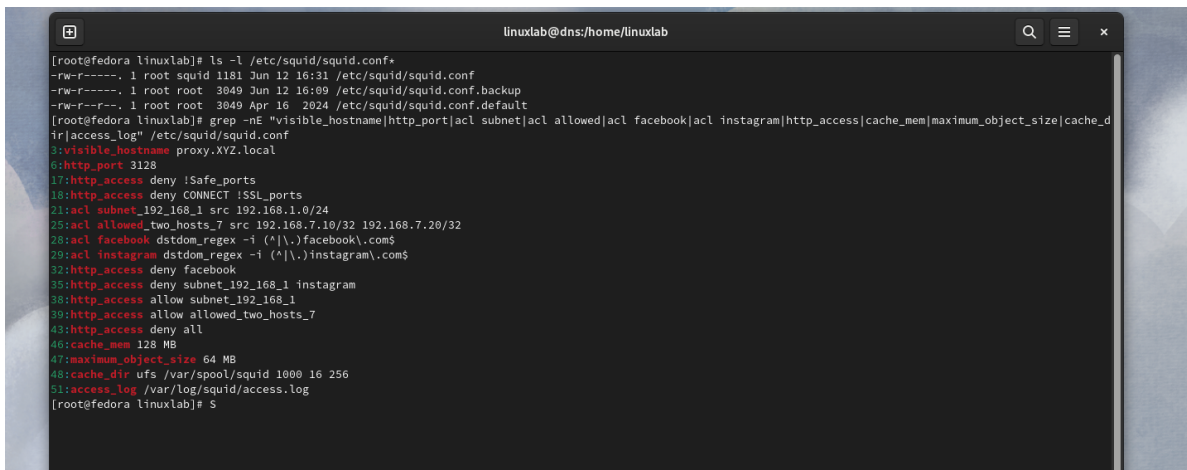
```
Upgraded:
  libstdc++-13.2.1-7.fc38.x86_64
Installed:
  libcap-1.0.1-12.fc38.x86_64      perl-DBI-1.643-15.fc38.x86_64      perl-Digest-SHA-1:6.04-1.fc38.x86_64      perl-English-1.11-495.fc38.noarch
  perl-Math-BigInt-1:1.9998.39-1.fc38.noarch  perl-Math-BigRat-0.2624-3.fc38.noarch  perl-Math-Complex-1.59-495.fc38.noarch  squid-7:6.9-1.fc38.x86_64
Complete!
[root@fedora linuxlab]# rpm -q squid
squid-6.9-1.fc38.x86_64
[root@fedora linuxlab]# sudo cp /etc/squid/squid.conf /etc/squid/squid.conf.backup
```

Figure 10: Squid installation completed successfully and the package version was verified.

5.2 Backing Up and Editing squid.conf

Before editing Squid settings, the original /etc/squid/squid.conf file was backed up. The edited configuration defined the Squid hostname, port, ACLs, website blocking rules, allowed clients, and cache settings.

```
cp /etc/squid/squid.conf /etc/squid/squid.conf.backup
ls -l /etc/squid/squid.conf*
```



```
linuxlab@dns:/home/linuxlab
[root@fedora linuxlab]# ls -l /etc/squid/squid.conf*
-rw-r-----. 1 root squid 1181 Jun 12 16:31 /etc/squid/squid.conf
-rw-r-----. 1 root root 3049 Jun 12 16:09 /etc/squid/squid.conf.backup
-rw-r-----. 1 root root 3049 Apr 16 2024 /etc/squid/squid.conf.default
[root@fedora linuxlab]# grep -nE "visible_hostname|http_port|acl subnet|acl allowed|acl facebook|acl instagram|http_access|cache_mem|maximum_object_size|cache_dir|access_log" /etc/squid/squid.conf
3:visible_hostname proxy.XYZ.local
6:http_port 3128
17:http_access deny !Safe_ports
18:http_access deny CONNECT !SSL_ports
21:acl subnet_192_168_1 src 192.168.1.0/24
22:acl allowed_two_hosts_7 src 192.168.7.10/32 192.168.7.20/32
28:acl facebook dstdom_regex -i (^\\.\\.facebook\\.com$
29:acl instagram dstdom_regex -i (^\\.\\.instagram\\.com$
32:http_access deny facebook
35:http_access deny subnet_192_168_1 instagram
38:http_access allow subnet_192_168_1
39:http_access allow allowed_two_hosts_7
43:http_access deny all
46:cache_mem 128 MB
47:maximum_object_size 64 MB
48:cache_dir ufs /var/spool/squid 1000 16 256
51:access_log /var/log/squid/access.log
[root@fedora linuxlab]#
```

Figure 11: Squid configuration backup and ACL/access-control configuration lines.

The main Squid ACL and access rules were configured as follows:

```
visible_hostname proxy.XYZ.local
http_port 3128
acl subnet_192_168_1 src 192.168.1.0/24
acl allowed_two_hosts_7 src 192.168.7.10/32 192.168.7.20/32
acl facebook dstdom_regex -i (^\\.\\.facebook\\.com$
acl instagram dstdom_regex -i (^\\.\\.instagram\\.com$
http_access deny facebook
http_access deny subnet_192_168_1 instagram
http_access allow subnet_192_168_1
http_access allow allowed_two_hosts_7
http_access deny all
cache_dir ufs /var/spool/squid 1000 16 256
```


5.3 Squid Rule Summary

Requirement	Implemented Squid Rule
Allow subnet 192.168.1.0/24	acl subnet_192_168_1 src 192.168.1.0/24 and http_access allow subnet_192_168_1
Allow only two hosts from 192.168.7.0/24	acl allowed_two_hosts_7 src 192.168.7.10/32 192.168.7.20/32
Deny all access to Facebook	acl facebook dstdom_regex ... and http_access deny facebook
Deny Instagram for 192.168.1.0/24	acl instagram ... and http_access deny subnet_192_168_1 instagram
Enable caching	cache_mem, maximum_object_size, cache_dir, and access_log

5.4 Syntax Validation, Service Startup, and Port Verification

The Squid configuration was checked using `squid -k parse`. After the configuration parsed successfully, Squid was enabled and started. The service status showed active (running), and the `ss` command confirmed that Squid was listening on TCP port 3128.

```
squid -k parse
systemctl enable --now squid
systemctl status squid
ss -tulnp | grep 3128
```

```
[root@fedora linuxlab]# nano /etc/squid/squid.conf
[root@fedora linuxlab]# sudo squid -k parse
2026/06/12 16:16:01| Processing Configuration File: /etc/squid/squid.conf (depth 0)
2026/06/12 16:16:01| Processing: visible_hostname proxy.XYZ.local
2026/06/12 16:16:01| Processing: http_port 3128
2026/06/12 16:16:01| Processing: acl SSL_ports port 443
2026/06/12 16:16:01| Processing: acl Safe_ports port 80
2026/06/12 16:16:01| Processing: acl Safe_ports port 443
2026/06/12 16:16:01| Processing: acl Safe_ports port 21
2026/06/12 16:16:01| Processing: acl Safe_ports port 1025-65535
2026/06/12 16:16:01| Processing: acl CONNECT method CONNECT
2026/06/12 16:16:01| Processing: http_access deny !Safe_ports
2026/06/12 16:16:01| Processing: http_access deny CONNECT !SSL_ports
2026/06/12 16:16:01| Processing: acl subnet_192_168_1 src 192.168.1.0/24
2026/06/12 16:16:01| Processing: acl allowed_two_hosts_7 src 192.168.7.10/32 192.168.7.20/32
2026/06/12 16:16:01| Processing: acl facebook dstdomain facebook.com
2026/06/12 16:16:01| Processing: acl instagram dstdomain instagram.com
2026/06/12 16:16:01| Processing: http_access deny facebook
2026/06/12 16:16:01| Processing: http_access deny subnet_192_168_1 instagram
2026/06/12 16:16:01| Processing: http_access allow subnet_192_168_1
2026/06/12 16:16:01| Processing: http_access allow allowed_two_hosts_7
2026/06/12 16:16:01| Processing: http_access deny all
2026/06/12 16:16:01| Processing: cache_mem 128 MB
2026/06/12 16:16:01| Processing: maximum_object_size 64 MB
2026/06/12 16:16:01| Processing: cache_dir ufs /var/spool/squid 1000 16 256
2026/06/12 16:16:01| Processing: access_log /var/log/squid/access.log
```

Figure 12: Squid configuration syntax validation using `squid -k parse`.

```
[root@fedora linuxlab]# sudo systemctl enable --now squid
Created symlink /etc/systemd/system/multi-user.target.wants/squid.service → /usr/lib/systemd/system/squid.service.
[root@fedora linuxlab]# sudo systemctl restart squid

[root@fedora linuxlab]#
[root@fedora linuxlab]# sudo systemctl status squid
● squid.service - Squid caching proxy
   Loaded: loaded (/usr/lib/systemd/system/squid.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─10-timeout-abort.conf
   Active: active (running) since Fri 2026-06-12 16:18:08 EEST; 20s ago
     Docs: man:squid(8)
   Process: 3654 ExecStartPre=/usr/libexec/squid/cache_swap.sh (code=exited, status=0/SUCCESS)
    Main PID: 3657 (squid)
       Tasks: 3 (limit: 8224)
      Memory: 14.0M
         CPU: 91ms
    CGroup: /system.slice/squid.service
            └─3657 /usr/sbin/squid --foreground -f /etc/squid/squid.conf
              └─3659 "(squid-1)" --kid squid-1 --foreground -f /etc/squid/squid.conf
                └─3660 "(unlinkd)"

Jun 12 16:18:08 fedora systemd[1]: Starting squid.service - Squid caching proxy...
Jun 12 16:18:08 fedora squid[3657]: Squid Parent: will start 1 kids
Jun 12 16:18:08 fedora squid[3657]: Squid Parent: (squid-1) process 3659 started
Jun 12 16:18:08 fedora systemd[1]: Started squid.service - Squid caching proxy.
[root@fedora linuxlab]# firewall-cmd --permanent --add-port=3128/tcp
Warning: ALREADY_ENABLED: 3128:tcp
success
[root@fedora linuxlab]# firewall-cmd --reload
success
[root@fedora linuxlab]# firewall-cmd --list-ports
1025-65535/tcp 1025-65535/udp
[root@fedora linuxlab]# ss -tlnp | grep 3128
tcp LISTEN 0      4096          *:3128          :* users:((("squid",pid=3659,fd=16))
[root@fedora linuxlab]# curl -I -x http://192.168.1.200:3128 http://example.com
HTTP/1.1 200 OK
Date: Fri, 12 Jun 2026 13:19:46 GMT
Content-Type: text/html
Server: cloudflare
Last-Modified: Thu, 11 Jun 2026 20:55:20 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
```

Figure 13: Squid service running, port 3128 listening, and example.com proxy access test.

5.5 Firewall Verification

The active FedoraWorkstation firewall zone was checked. Although the list-ports output showed the broader allowed range 1025-65535/tcp, the direct query confirmed that port 3128/tcp was allowed.

```
firewall-cmd --get-active-zones
firewall-cmd --list-all
firewall-cmd --query-port=3128/tcp
```

```
[root@fedora linuxlab]# firewall-cmd --get-active-zones
FedoraWorkstation
  interfaces: enp0s3
[root@fedora linuxlab]# firewall-cmd --list-all
FedoraWorkstation (active)
  target: default
  icmp-block-inversion: no
  interfaces: enp0s3
  sources:
  services: dhcpv6-client dns mdns samba-client ssh
  ports: 1025-65535/udp 1025-65535/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[root@fedora linuxlab]# firewall-cmd --list-ports --permanent
1025-65535/tcp 1025-65535/udp
[root@fedora linuxlab]# firewall-cmd --add-port=3128/tcp
firewall-cmd --permanent --add-port=3128/tcp
firewall-cmd --reload
firewall-cmd --list-ports
Warning: ALREADY_ENABLED: '3128:tcp' already in 'FedoraWorkstation'
success
Warning: ALREADY_ENABLED: 3128:tcp
success
success
1025-65535/tcp 1025-65535/udp
[root@fedora linuxlab]# firewall-cmd --get-active-zones
FedoraWorkstation
  interfaces: enp0s3
[root@fedora linuxlab]# firewall-cmd --query-port=3128/tcp
yes
[root@fedora linuxlab]# curl -I -x http://192.168.1.200:3128 http://example.com
HTTP/1.1 200 OK
Date: Fri, 12 Jun 2026 13:19:46 GMT
Content-Type: text/html
Server: cloudflare
```

Figure 14: Firewall zone information and query confirming port 3128/tcp is allowed.

5.6 Proxy Access Tests Using curl

The proxy was tested with curl using the -x option. example.com returned HTTP/1.1 200 OK, proving allowed access through the proxy. Facebook and Instagram returned HTTP/1.1 403 Forbidden, proving that the Squid deny rules were working.

```
curl -I -x http://192.168.1.200:3128 http://example.com
curl -I -x http://192.168.1.200:3128 http://www.facebook.com
curl -I -x http://192.168.1.200:3128 http://www.instagram.com
```

```
2020/06/12 10:31:29] Requiring client certificates.
[root@fedora linuxlab]# curl -I -x http://192.168.1.200:3128 http://www.facebook.com
curl -I -x http://192.168.1.200:3128 http://www.instagram.com
HTTP/1.1 403 Forbidden
Server: squid/6.9
Mime-Version: 1.0
Date: Fri, 12 Jun 2020 13:33:08 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 3512
X-Squid-Error: ERR_ACCESS_DENIED 0
Vary: Accept-Language
Content-Language: en
Cache-Status: proxy.XYZ.local
Via: 1.1 proxy.XYZ.local (squid/6.9)
Connection: keep-alive

HTTP/1.1 403 Forbidden
Server: squid/6.9
Mime-Version: 1.0
Date: Fri, 12 Jun 2020 13:33:08 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 3515
X-Squid-Error: ERR_ACCESS_DENIED 0
Vary: Accept-Language
Content-Language: en
Cache-Status: proxy.XYZ.local
Via: 1.1 proxy.XYZ.local (squid/6.9)
Connection: keep-alive

[root@fedora linuxlab]#
```

Figure 15: Curl tests showing Facebook and Instagram blocked with HTTP 403 Forbidden.

5.7 Browser Proxy Test

Firefox was configured manually to use the Squid server at 192.168.1.200 on port 3128. The browser tests showed the Squid error page for Facebook and Instagram, confirming practical proxy enforcement in a graphical client.

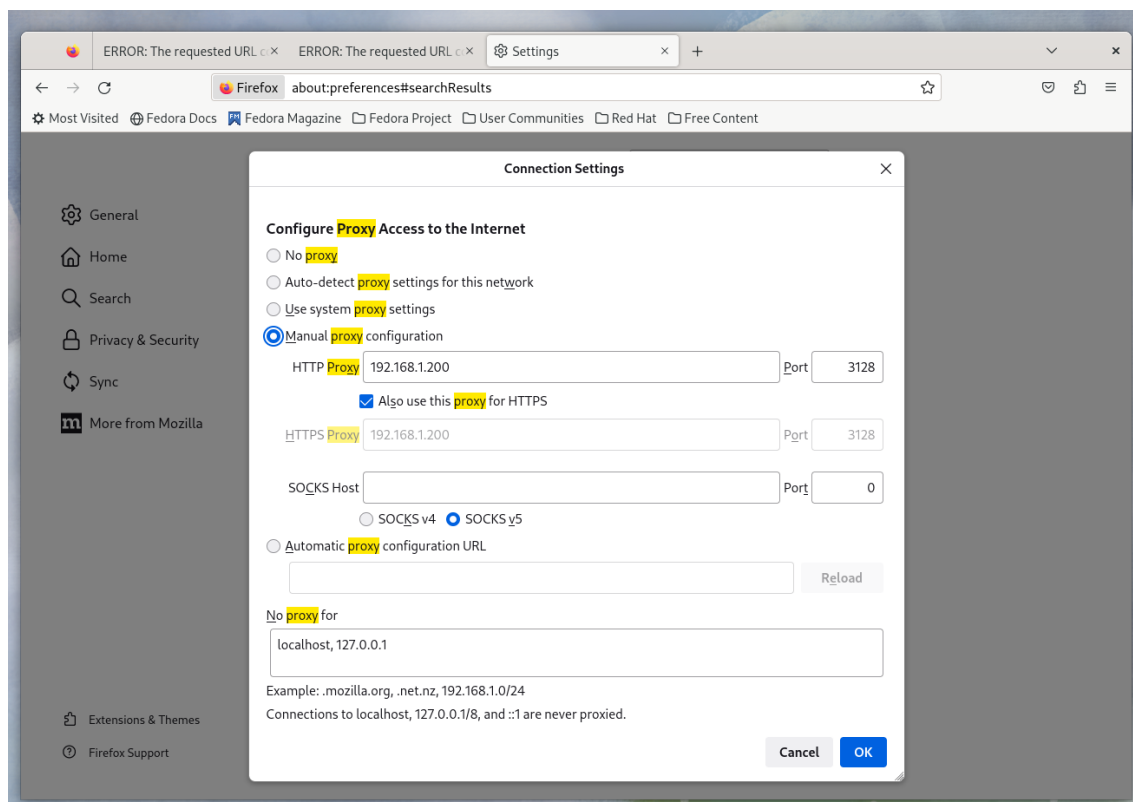


Figure 16: Firefox manual proxy configuration using 192.168.1.200:3128.

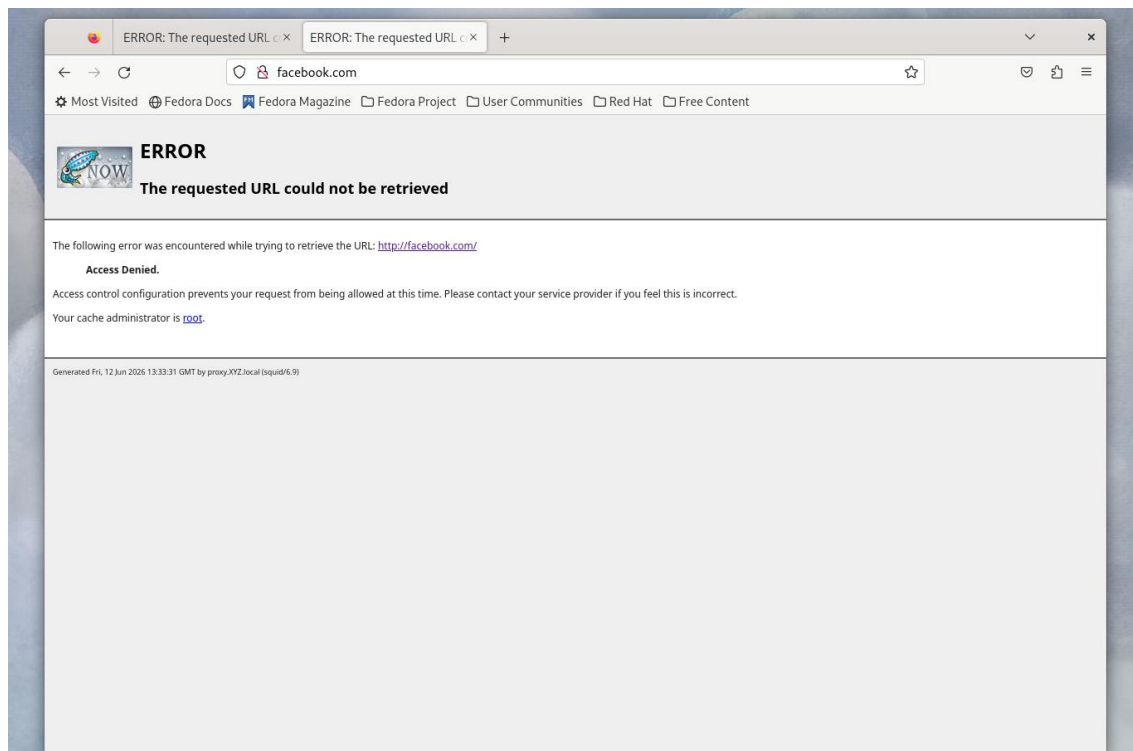


Figure 17: Browser test showing facebook.com denied by Squid.

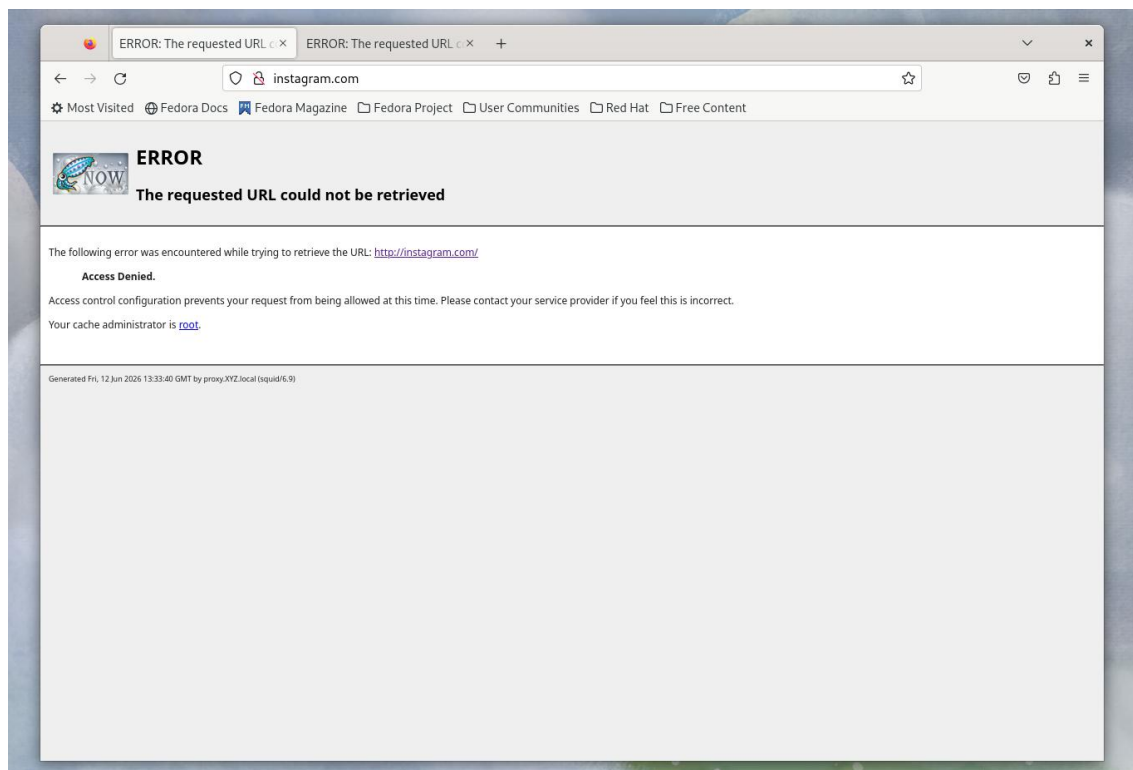


Figure 18: Browser test showing *instagram.com* denied by Squid.

6. Squid Caching Demonstration

To prove caching clearly, a controlled local HTTP server was used instead of relying on modern HTTPS websites. A 10 MB test file was created and served by a Python web server on port 8000 with cache-friendly headers. The file was requested twice through Squid. This avoids confusion caused by browser caching, encrypted HTTPS traffic, or websites that disable proxy caching.

```
mkdir -p ~/cache_origin
cd ~/cache_origin
dd if=/dev/urandom of=cache-test.bin bs=1M count=10
python3 cache_server.py
```

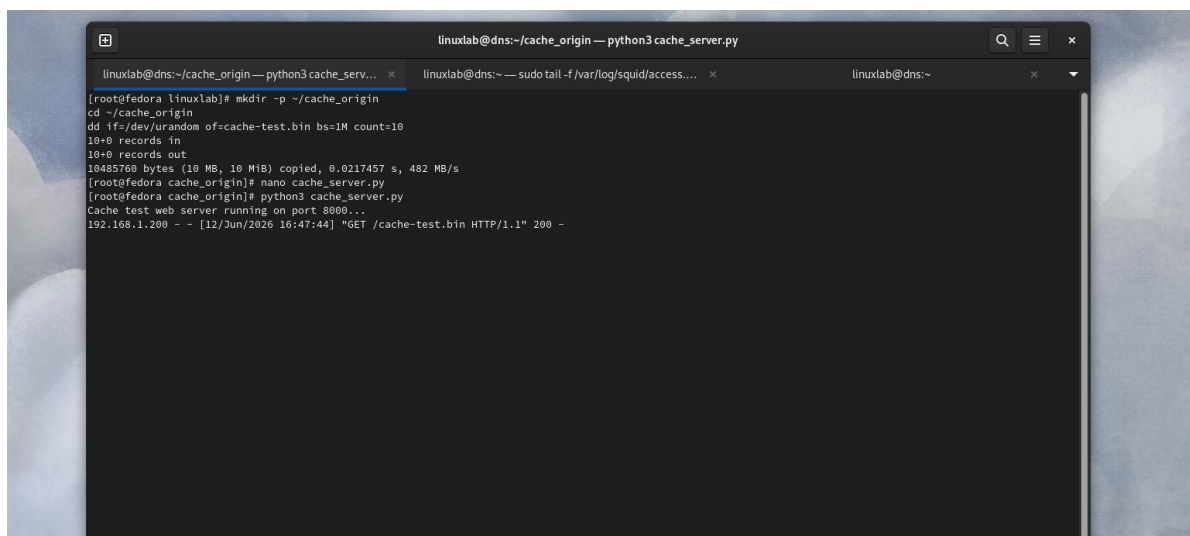


Figure 19: Local Python origin server used to provide a cacheable test file for Squid caching verification.

The Squid access log was cleared and watched while the same file was requested twice through the proxy. The first request appeared as TCP_MISS/200, meaning Squid fetched the file from the origin server. The second request appeared as TCP_HIT/200, meaning Squid served the file from cache. HIER_DIRECT on the first request and HIER_NONE on the second request further confirmed that the origin server was not contacted again for the cached response.

```
sudo truncate -s 0 /var/log/squid/access.log
sudo tail -f /var/log/squid/access.log
curl --no-proxy "" -o /dev/null -x http://192.168.1.200:3128 http://192.168.1.200:8000/cache-test.bin
```

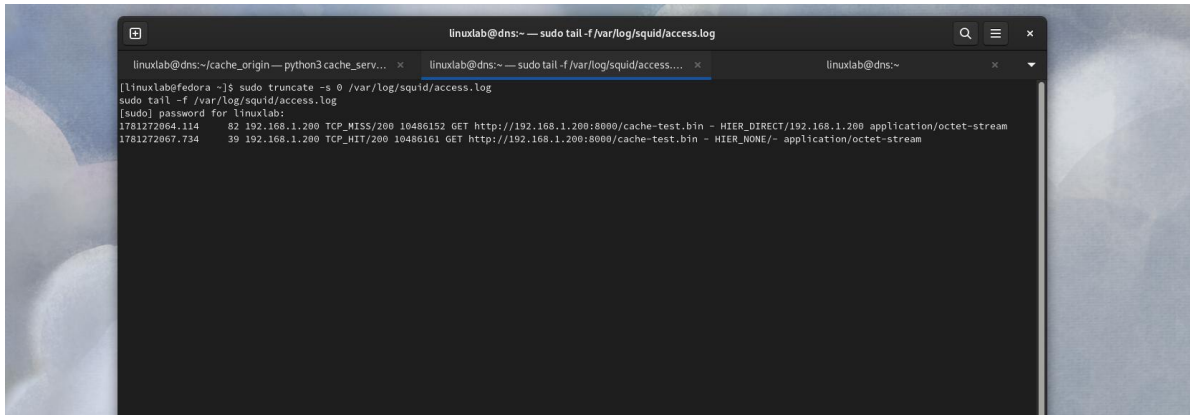


Figure 20: Squid access log showing TCP_MISS on the first request and TCP_HIT on the second request.

The repeated curl requests also showed an improved transfer speed on the second request, supporting the cache hit result shown in the Squid log.

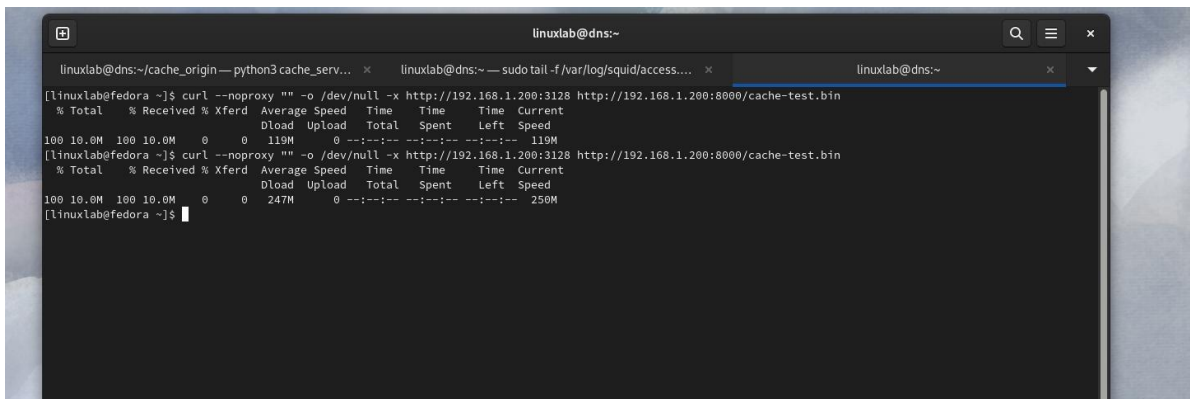


Figure 21: Repeated curl requests through Squid showing improved transfer speed after caching.

7. Conclusion

The Fedora Linux virtual machine was successfully configured to provide both DNS and proxy services. BIND was installed and configured as an authoritative DNS server for the XYZ.local domain with both forward and reverse lookup zones. The DNS server correctly resolved all required hostnames and reverse IP records using nslookup. Squid was installed and configured as a proxy server with ACL rules to allow specific networks and hosts, block Facebook for all users, and block Instagram for the 192.168.1.0/24 subnet. Proxy testing through curl and Firefox confirmed that allowed traffic passed through the proxy and denied websites were blocked. Finally, Squid caching was demonstrated using a controlled test file where the first request showed TCP_MISS and the repeated request showed TCP_HIT, proving that Squid caching was functioning correctly.